计算机体系结构
实验 2
乘除、分支跳转与访存指令的添加

鲁昕宁　2414015

2026 年 10 月 23 日

GitHub 仓库　SidneyLu/Computer-Architecture-NKU-CS/tree/master/lab2

班号　0964

授课教师　董前琨

1 实验目标

本实验接着实验 1 带前递的五级流水线，完成《CPU 设计实战》第 5 章和第 6 章。CPU 仍是 mycpu_verify/rtl/myCPU。

1. 补齐算术、逻辑、变量移位，以及 MULT/MULTU/DIV/DIVU 和 HI/LO 的读写，使 func_lab6 的 42 个功能点与参考轨迹一致。
2. 补齐 J、条件分支、JALR，以及字节、半字和非对齐的 LWL/LWR/SWL/SWR，使 func_lab7 的 68 个功能点通过。

乘法和除法用 Vivado 的 mult_gen、div_gen。前递和 load-use 停顿沿用实验 1。没有实验箱，结论以仿真波形和 trace 对比为准。

2 实验原理

流水线、阻塞和前递已经写在实验 1 里。下面是这次新增的立即数扩展、HI/LO 和按字节访问存储器。

2.1 两种立即数扩展

ADDI、SLTI、SLTIU、访存偏移和 LUI 的低 16 位按符号扩展。ANDI、ORI、XORI 单独零扩展。逻辑立即数最高位为 1 时，符号扩展会把高 16 位填成 ffff。

SLTIU 的立即数仍做符号扩展，比较按无符号进行。负数扩展后接近 2^{32}，无符号比较时大于绝大多数 32 位操作数。

2.2 HI/LO 与多周期乘除

MULT/MULTU 把 64 位积的高半写入 HI、低半写入 LO，DIV/DIVU 把商写入 LO、余数写入 HI，余数的符号与被除数相同。这四条指令不写通用寄存器。MFHI/MFLO 把 HI 或 LO 当作该指令的写回数据，MTHI/MTLO 用 rs 覆盖 HI 或 LO。

乘法器流水级数为 1，结果再打一拍后采样，所以一条乘法在执行级占两拍。除法器有符号延迟 36 拍、无符号延迟 34 拍。结果未到时 es_ready_go 为 0，后续指令留在译码级。除法器的 tvalid 只在进入执行级后的一拍拉高；每个时钟都保持为 1 时，除法会重新开始。测试程序不覆盖溢出例外和除零。

2.3 部分字与非对齐拼接

数据 RAM 按字寻址，addr[17:2] 选字，addr[1:0] 选字节。小端下 wen[0] 对应 [7:0]。LB/LBU/LH/LHU 在访存级抽出字节或半字再扩展，旁路出去的是扩展后的值。SB/SH 只打开对应的写使能，写数据把同一个字节或半字复制到整个字上，写使能关掉的字节保持原值。

LWL/LWR 用 rt 的旧值和内存中的字拼接。旧值只在译码级读一次（含前递），然后锁存到访存级。小端、地址低两位为 k 时：

k	LWL	LWR
0	{mem[7:0], rt[23:0]}	mem
1	{mem[15:0], rt[15:0]}	{rt[31:24], mem[31:8]}
2	{mem[23:0], rt[7:0]}	{rt[31:16], mem[31:16]}
3	mem	{rt[31:8], mem[31:24]}

BGEZAL/BLTZAL 无论条件是否成立都写 $31 = PC+8。条件只决定下一条（延迟槽）之后从哪里取指。

3 实验过程

3.1 IP 与存储器

乘法器用两个 mult_gen：有符号和无符号各一个，32×32，LUT，流水级数 1，端口是 CLK、A、B、P[63:0]。除法器用两个 div_gen，Radix2，操作数 32 位，输出带余数，不检测除零，没有 tready 和复位。有符号延迟 36，无符号延迟 34。商在 tdata[63:32]，余数在 tdata[31:0]。乘法器则是高 32 位进 HI、低 32 位进 LO。

inst_ram.vhd 是 IP 生成的只读文件。换程序要改 inst_ram.xci 里的 COE，并对锁定的旧 IP 先 upgrade_ip，再重新生成仿真文件。func_lab6 与 func_lab7 的 coe 放在 CPU_CDE/soft/。cpu132_gettrace 跑完后得到 golden_trace.txt，mycpu 的 testbench 按写回逐条比较。

3.2 func_lab6

42 个功能点一次通过。同一序号上 mycpu 都更早到达：乘除期间执行级停等，其余指令沿用实验 1 的前递，整段仿真短于参考 CPU。逐点记录见第 5.1 节。

3.3 func_lab7 与 LWR

换成 func_lab7 的 COE 后，第 1 到 63 点通过，第 64 点 LWR 第一次不一致：

```
reference: PC = 0xbfc9a264, wnum = 0x02, wdata = 0xca5c0495
mycpu    : PC = 0xbfc9a264, wnum = 0x02, wdata = 0xf15c0495
```

该指令是 lwr $v0,-16999($at)，地址低两位为 1。内存字是 5c049582，$v0 的旧值是 ca18f110。正确拼接是 {rt[31:24], mem[31:8]} = ca5c0495。原先把 rt[31:8] 与 mem[31:8] 接成 48 位，Verilog 截断后高字节变成 f1。按上表的位宽改完后重新仿真，68 个功能点通过，记录见第 5.1 节。延迟槽测试在这次修改之前已经通过，分支目标仍用实验 1 的 fs_pc + offset。

4 关键代码

下面各段对应译码控制、HI/LO、模块级封装和部分字访存。

4.1 译码：扩展、链接和乘除

零扩展单独用 src2_is_zimm。链接类指令把 PC 和常数 8 送进 ALU，目标寄存器是 $31；JALR 用 rd。乘除和 MTHI/MTLO 不写通用寄存器。

```
assign src2_is_imm  = inst_addiu | inst_addi | inst_slti | inst_sltiu
                    | inst_lui | inst_load | inst_store;
assign src2_is_zimm = inst_andi | inst_ori | inst_xori;
assign src1_is_pc   = inst_jal | inst_jalr | inst_bgezal | inst_bltzal;
assign src2_is_8    = inst_jal | inst_jalr | inst_bgezal | inst_bltzal;
assign dst_is_r31   = inst_jal | inst_bgezal | inst_bltzal;
assign gr_we        = ~(inst_store | inst_beq | inst_bne | inst_jr | inst_j
                      | inst_bgez | inst_bgtz | inst_blez | inst_bltz
                      | inst_mult | inst_multu | inst_div | inst_divu
                      | inst_mthi | inst_mtlo);
```

BLTZAL 的条件是 rs[31]，BGEZ 是 ~rs[31]。条件不成立时 br_taken 为 0，但 gr_we 仍为 1，$31 照常写回。

4.2 HI/LO

乘法器输出打一拍后采样。除法只在进入执行级的下一拍拉高一次 tvalid。商进 LO，余数进 HI。MFLO 的写回数据取 LO。

```
assign es_ready_go = (es_is_mul | es_is_divop) ? md_ready : 1'b1;


if (es_valid && es_is_divop && div_accepted && div_vout && !md_ready) begin
    lo       <= div_dout[63:32];
    hi       <= div_dout[31:0];
    md_ready <= 1'b1;
end
if (es_valid && es_is_mul && !md_ready) begin
    if (!mul_seen) mul_seen <= 1'b1;
    else begin
        hi <= mul_res[63:32];
        lo <= mul_res[31:0];
        md_ready <= 1'b1;
    end
end
assign es_wb_data = es_is_mfhi ? hi : es_is_mflo ? lo : alu_result;
```

4.3 模块级封装

mul_tb、div_tb 的端口与 IP 不同。mul.v、div.v 把同一组核接到测试台上。两个乘法器每个时钟都在计算，mul_signed 打一拍后再选择结果，与 IP 的输出寄存器对齐。复位期间 result 保持 0。

```
always @(posedge mul_clk) begin
    if (!resetn) begin
        en       <= 1'b0;
        signed_r <= 1'b0;
    end
    else begin
        en       <= 1'b1;
        signed_r <= mul_signed;
    end
end
assign result = (!en)    ? 64'd0 :
                signed_r ? p_signed :
                           p_unsigned;
```

除法在 div 为 1 且尚未接受时，把 fire 拉高一拍，只送给 div_signed 选中的核。signed_hold 记下符号，complete 取该核的 dout_tvalid。商在 [63:32]，余数在 [31:0]。

```
assign fire = resetn && div && !accepted;

div_gen_signed_0 u_signed (
    .s_axis_divisor_tvalid  (fire && div_signed),
    .s_axis_dividend_tvalid (fire && div_signed),
    ...
);
div_gen_unsigned_0 u_unsigned (
    .s_axis_divisor_tvalid  (fire && !div_signed),
    .s_axis_dividend_tvalid (fire && !div_signed),
    ...
);
assign s        = dout[63:32];
assign r        = dout[31:0];
assign complete = signed_hold ? vout_s : vout_u;

always @(posedge div_clk) begin
    if (!resetn) begin
        accepted    <= 1'b0;
        signed_hold <= 1'b0;
    end
    else if (!div)
        accepted <= 1'b0;
```

```
    else if (!accepted) begin
        accepted    <= 1'b1;
        signed_hold <= div_signed;
    end
end
```

4.4 字节写使能与 LWR

```
assign st_wen = (es_mem_op == `MEM_SW) ? 4'b1111 :
                (es_mem_op == `MEM_SB) ? (4'b0001 << ea) :
                (es_mem_op == `MEM_SH) ? (ea[1] ? 4'b1100 : 4'b0011) :
                ...
assign data_sram_wen = es_valid ? st_wen : 4'h0;

(ms_mem_op == `MEM_LWR && ea == 2'd1) ? {ms_rt_value[31:24], mem_word[31:8]}
↪  :
(ms_mem_op == `MEM_LWR && ea == 2'd2) ? {ms_rt_value[31:16],
↪  mem_word[31:16]} :
(ms_mem_op == `MEM_LWR && ea == 2'd3) ? {ms_rt_value[31:8],
↪  mem_word[31:24]} :
```

`ea == 0` 的 LWR 以及 LWL 的 `ea == 3` 直接取 `mem_word`。

5 实验结果

5.1 功能点对比

仿真控制台每隔约 10 µs 打印一行 `Test is running`。下面保留功能点通过行和结束行。左栏是 `cpu132_gettrace`，右栏是 `mycpu`，同一行是同一个功能点。`gettrace` 结束时打印 `gettrace end!`，表示 `golden_trace.txt` 已经写好；`mycpu` 结束时打印 `----PASS!!!`，表示写回与这条轨迹一致。

`func_lab6` 两边都到第 42 点。`gettrace` 在 1194255 ns 结束，`mycpu` 在 1077765 ns 结束。

gettrace

```
Test begin!
----[  14055 ns] Number 8'd01 Functional Test Point PASS!!!
----[  41545 ns] Number 8'd02 Functional Test Point PASS!!!
----[  63015 ns] Number 8'd03 Functional Test Point PASS!!!
----[  90505 ns] Number 8'd04 Functional Test Point PASS!!!
----[ 117995 ns] Number 8'd05 Functional Test Point PASS!!!
----[ 145485 ns] Number 8'd06 Functional Test Point PASS!!!
----[ 172975 ns] Number 8'd07 Functional Test Point PASS!!!
----[ 200465 ns] Number 8'd08 Functional Test Point PASS!!!
----[ 227955 ns] Number 8'd09 Functional Test Point PASS!!!
----[ 255445 ns] Number 8'd10 Functional Test Point PASS!!!
----[ 276915 ns] Number 8'd11 Functional Test Point PASS!!!
----[ 298385 ns] Number 8'd12 Functional Test Point PASS!!!
----[ 319855 ns] Number 8'd13 Functional Test Point PASS!!!
----[ 333695 ns] Number 8'd14 Functional Test Point PASS!!!
----[ 348585 ns] Number 8'd15 Functional Test Point PASS!!!
----[ 420255 ns] Number 8'd16 Functional Test Point PASS!!!
----[ 496875 ns] Number 8'd17 Functional Test Point PASS!!!
----[ 559275 ns] Number 8'd18 Functional Test Point PASS!!!
----[ 613675 ns] Number 8'd19 Functional Test Point PASS!!!
----[ 615745 ns] Number 8'd20 Functional Test Point PASS!!!
----[ 617815 ns] Number 8'd21 Functional Test Point PASS!!!
----[ 619425 ns] Number 8'd22 Functional Test Point PASS!!!
----[ 621035 ns] Number 8'd23 Functional Test Point PASS!!!
----[ 643935 ns] Number 8'd24 Functional Test Point PASS!!!
----[ 665405 ns] Number 8'd25 Functional Test Point PASS!!!
----[ 687855 ns] Number 8'd26 Functional Test Point PASS!!!
----[ 709325 ns] Number 8'd27 Functional Test Point PASS!!!
----[ 730795 ns] Number 8'd28 Functional Test Point PASS!!!
----[ 752265 ns] Number 8'd29 Functional Test Point PASS!!!
----[ 773735 ns] Number 8'd30 Functional Test Point PASS!!!
----[ 795205 ns] Number 8'd31 Functional Test Point PASS!!!
----[ 822695 ns] Number 8'd32 Functional Test Point PASS!!!
----[ 850185 ns] Number 8'd33 Functional Test Point PASS!!!
----[ 877675 ns] Number 8'd34 Functional Test Point PASS!!!
----[ 935355 ns] Number 8'd35 Functional Test Point PASS!!!
----[1019275 ns] Number 8'd36 Functional Test Point PASS!!!
----[1093745 ns] Number 8'd37 Functional Test Point PASS!!!
----[1160535 ns] Number 8'd38 Functional Test Point PASS!!!
----[1170115 ns] Number 8'd39 Functional Test Point PASS!!!
----[1179675 ns] Number 8'd40 Functional Test Point PASS!!!
----[1188615 ns] Number 8'd41 Functional Test Point PASS!!!
----[1193795 ns] Number 8'd42 Functional Test Point PASS!!!
============================================================
gettrace end!
----Succeed in generating trace file!
$finish called at time : 1194255 ns : File "D:/Architecture/lab1/
↪  lab3/CPU_CDE/cpu132_gettrace/testbench/tb_top.v" Line 221
```

mycpu

```
Test begin!
----[  14045 ns] Number 8'd01 Functional Test Point PASS!!!
----[  41495 ns] Number 8'd02 Functional Test Point PASS!!!
----[  62925 ns] Number 8'd03 Functional Test Point PASS!!!
----[  90375 ns] Number 8'd04 Functional Test Point PASS!!!
----[ 117825 ns] Number 8'd05 Functional Test Point PASS!!!
----[ 145275 ns] Number 8'd06 Functional Test Point PASS!!!
----[ 172725 ns] Number 8'd07 Functional Test Point PASS!!!
----[ 200175 ns] Number 8'd08 Functional Test Point PASS!!!
----[ 227625 ns] Number 8'd09 Functional Test Point PASS!!!
----[ 255075 ns] Number 8'd10 Functional Test Point PASS!!!
----[ 276505 ns] Number 8'd11 Functional Test Point PASS!!!
----[ 297935 ns] Number 8'd12 Functional Test Point PASS!!!
----[ 319365 ns] Number 8'd13 Functional Test Point PASS!!!
----[ 330605 ns] Number 8'd14 Functional Test Point PASS!!!
----[ 342695 ns] Number 8'd15 Functional Test Point PASS!!!
----[ 407295 ns] Number 8'd16 Functional Test Point PASS!!!
----[ 475855 ns] Number 8'd17 Functional Test Point PASS!!!
----[ 532215 ns] Number 8'd18 Functional Test Point PASS!!!
----[ 580575 ns] Number 8'd19 Functional Test Point PASS!!!
----[ 582455 ns] Number 8'd20 Functional Test Point PASS!!!
----[ 584335 ns] Number 8'd21 Functional Test Point PASS!!!
----[ 585745 ns] Number 8'd22 Functional Test Point PASS!!!
----[ 587165 ns] Number 8'd23 Functional Test Point PASS!!!
----[ 610025 ns] Number 8'd24 Functional Test Point PASS!!!
----[ 631455 ns] Number 8'd25 Functional Test Point PASS!!!
----[ 653865 ns] Number 8'd26 Functional Test Point PASS!!!
----[ 675295 ns] Number 8'd27 Functional Test Point PASS!!!
----[ 696725 ns] Number 8'd28 Functional Test Point PASS!!!
----[ 718155 ns] Number 8'd29 Functional Test Point PASS!!!
----[ 739585 ns] Number 8'd30 Functional Test Point PASS!!!
----[ 761015 ns] Number 8'd31 Functional Test Point PASS!!!
----[ 788465 ns] Number 8'd32 Functional Test Point PASS!!!
----[ 815915 ns] Number 8'd33 Functional Test Point PASS!!!
----[ 843365 ns] Number 8'd34 Functional Test Point PASS!!!
----[ 905885 ns] Number 8'd35 Functional Test Point PASS!!!
----[ 993325 ns] Number 8'd36 Functional Test Point PASS!!!
----[1020055 ns] Number 8'd37 Functional Test Point PASS!!!
----[1044205 ns] Number 8'd38 Functional Test Point PASS!!!
----[1053745 ns] Number 8'd39 Functional Test Point PASS!!!
----[1063265 ns] Number 8'd40 Functional Test Point PASS!!!
----[1072165 ns] Number 8'd41 Functional Test Point PASS!!!
----[1077305 ns] Number 8'd42 Functional Test Point PASS!!!
============================================================
Test end!
----PASS!!!
$finish called at time : 1077765 ns : File "D:/Architecture/lab1/
↪  lab3/CPU_CDE/mycpu_verify/testbench/mycpu_tb.v" Line 267
```

`func_lab7` 两边都到第 68 点。`gettrace` 在 1835885 ns 结束，`mycpu` 在 1645135 ns 结束。第 42 点仍是 1077305 ns，与单独运行 `func_lab6` 相同。前 34 点两边相差不超过 34.3 µs。除法所在的第 35、36 点耗时接近；第 37 点乘法之后差距拉开，到第 42 点为 0.116 ms，到第 68 点为 0.191 ms。

gettrace

```
Test begin!
----[  14055 ns] Number 8'd01 Functional Test Point PASS!!!
----[  41545 ns] Number 8'd02 Functional Test Point PASS!!!
----[  63015 ns] Number 8'd03 Functional Test Point PASS!!!
----[  90505 ns] Number 8'd04 Functional Test Point PASS!!!
----[ 117995 ns] Number 8'd05 Functional Test Point PASS!!!
----[ 145485 ns] Number 8'd06 Functional Test Point PASS!!!
----[ 172975 ns] Number 8'd07 Functional Test Point PASS!!!
----[ 200465 ns] Number 8'd08 Functional Test Point PASS!!!
----[ 227955 ns] Number 8'd09 Functional Test Point PASS!!!
----[ 255445 ns] Number 8'd10 Functional Test Point PASS!!!
----[ 276915 ns] Number 8'd11 Functional Test Point PASS!!!
----[ 298385 ns] Number 8'd12 Functional Test Point PASS!!!
----[ 319855 ns] Number 8'd13 Functional Test Point PASS!!!
----[ 333695 ns] Number 8'd14 Functional Test Point PASS!!!
----[ 348585 ns] Number 8'd15 Functional Test Point PASS!!!
----[ 420255 ns] Number 8'd16 Functional Test Point PASS!!!
----[ 496875 ns] Number 8'd17 Functional Test Point PASS!!!
----[ 559275 ns] Number 8'd18 Functional Test Point PASS!!!
----[ 613675 ns] Number 8'd19 Functional Test Point PASS!!!
----[ 615745 ns] Number 8'd20 Functional Test Point PASS!!!
----[ 617815 ns] Number 8'd21 Functional Test Point PASS!!!
----[ 619425 ns] Number 8'd22 Functional Test Point PASS!!!
----[ 621035 ns] Number 8'd23 Functional Test Point PASS!!!
----[ 643935 ns] Number 8'd24 Functional Test Point PASS!!!
----[ 665405 ns] Number 8'd25 Functional Test Point PASS!!!
----[ 687855 ns] Number 8'd26 Functional Test Point PASS!!!
----[ 709325 ns] Number 8'd27 Functional Test Point PASS!!!
----[ 730795 ns] Number 8'd28 Functional Test Point PASS!!!
----[ 752265 ns] Number 8'd29 Functional Test Point PASS!!!
----[ 773735 ns] Number 8'd30 Functional Test Point PASS!!!
----[ 795205 ns] Number 8'd31 Functional Test Point PASS!!!
----[ 822695 ns] Number 8'd32 Functional Test Point PASS!!!
----[ 850185 ns] Number 8'd33 Functional Test Point PASS!!!
----[ 877675 ns] Number 8'd34 Functional Test Point PASS!!!
----[ 935355 ns] Number 8'd35 Functional Test Point PASS!!!
----[1019275 ns] Number 8'd36 Functional Test Point PASS!!!
----[1093745 ns] Number 8'd37 Functional Test Point PASS!!!
----[1160535 ns] Number 8'd38 Functional Test Point PASS!!!
----[1170115 ns] Number 8'd39 Functional Test Point PASS!!!
----[1179675 ns] Number 8'd40 Functional Test Point PASS!!!
----[1188615 ns] Number 8'd41 Functional Test Point PASS!!!
----[1193795 ns] Number 8'd42 Functional Test Point PASS!!!
----[1222025 ns] Number 8'd43 Functional Test Point PASS!!!
----[1265245 ns] Number 8'd44 Functional Test Point PASS!!!
----[1308185 ns] Number 8'd45 Functional Test Point PASS!!!
----[1351405 ns] Number 8'd46 Functional Test Point PASS!!!
----[1385005 ns] Number 8'd47 Functional Test Point PASS!!!
----[1432805 ns] Number 8'd48 Functional Test Point PASS!!!
----[1482775 ns] Number 8'd49 Functional Test Point PASS!!!
----[1537775 ns] Number 8'd50 Functional Test Point PASS!!!
----[1539745 ns] Number 8'd51 Functional Test Point PASS!!!
----[1547845 ns] Number 8'd52 Functional Test Point PASS!!!
----[1556585 ns] Number 8'd53 Functional Test Point PASS!!!
----[1565005 ns] Number 8'd54 Functional Test Point PASS!!!
----[1573745 ns] Number 8'd55 Functional Test Point PASS!!!
----[1583125 ns] Number 8'd56 Functional Test Point PASS!!!
----[1591865 ns] Number 8'd57 Functional Test Point PASS!!!
----[1598255 ns] Number 8'd58 Functional Test Point PASS!!!
----[1627845 ns] Number 8'd59 Functional Test Point PASS!!!
----[1659605 ns] Number 8'd60 Functional Test Point PASS!!!
----[1682825 ns] Number 8'd61 Functional Test Point PASS!!!
----[1709545 ns] Number 8'd62 Functional Test Point PASS!!!
----[1724385 ns] Number 8'd63 Functional Test Point PASS!!!
----[1740065 ns] Number 8'd64 Functional Test Point PASS!!!
----[1771905 ns] Number 8'd65 Functional Test Point PASS!!!
----[1801825 ns] Number 8'd66 Functional Test Point PASS!!!
----[1818865 ns] Number 8'd67 Functional Test Point PASS!!!
----[1835425 ns] Number 8'd68 Functional Test Point PASS!!!
============================================================
gettrace end!
----Succeed in generating trace file!
$finish called at time : 1835885 ns : File "D:/Architecture/lab1/
↪  lab3/CPU_CDE/cpu132_gettrace/testbench/tb_top.v" Line 221
```

mycpu

```
Test begin!
----[  14045 ns] Number 8'd01 Functional Test Point PASS!!!
----[  41495 ns] Number 8'd02 Functional Test Point PASS!!!
----[  62925 ns] Number 8'd03 Functional Test Point PASS!!!
----[  90375 ns] Number 8'd04 Functional Test Point PASS!!!
----[ 117825 ns] Number 8'd05 Functional Test Point PASS!!!
----[ 145275 ns] Number 8'd06 Functional Test Point PASS!!!
----[ 172725 ns] Number 8'd07 Functional Test Point PASS!!!
----[ 200175 ns] Number 8'd08 Functional Test Point PASS!!!
----[ 227625 ns] Number 8'd09 Functional Test Point PASS!!!
----[ 255075 ns] Number 8'd10 Functional Test Point PASS!!!
----[ 276505 ns] Number 8'd11 Functional Test Point PASS!!!
----[ 297935 ns] Number 8'd12 Functional Test Point PASS!!!
----[ 319365 ns] Number 8'd13 Functional Test Point PASS!!!
----[ 330605 ns] Number 8'd14 Functional Test Point PASS!!!
----[ 342695 ns] Number 8'd15 Functional Test Point PASS!!!
----[ 407295 ns] Number 8'd16 Functional Test Point PASS!!!
----[ 475855 ns] Number 8'd17 Functional Test Point PASS!!!
----[ 532215 ns] Number 8'd18 Functional Test Point PASS!!!
----[ 580575 ns] Number 8'd19 Functional Test Point PASS!!!
----[ 582455 ns] Number 8'd20 Functional Test Point PASS!!!
----[ 584335 ns] Number 8'd21 Functional Test Point PASS!!!
----[ 585745 ns] Number 8'd22 Functional Test Point PASS!!!
----[ 587165 ns] Number 8'd23 Functional Test Point PASS!!!
----[ 610025 ns] Number 8'd24 Functional Test Point PASS!!!
----[ 631455 ns] Number 8'd25 Functional Test Point PASS!!!
----[ 653865 ns] Number 8'd26 Functional Test Point PASS!!!
----[ 675295 ns] Number 8'd27 Functional Test Point PASS!!!
----[ 696725 ns] Number 8'd28 Functional Test Point PASS!!!
----[ 718155 ns] Number 8'd29 Functional Test Point PASS!!!
----[ 739585 ns] Number 8'd30 Functional Test Point PASS!!!
----[ 761015 ns] Number 8'd31 Functional Test Point PASS!!!
----[ 788465 ns] Number 8'd32 Functional Test Point PASS!!!
----[ 815915 ns] Number 8'd33 Functional Test Point PASS!!!
----[ 843365 ns] Number 8'd34 Functional Test Point PASS!!!
----[ 905885 ns] Number 8'd35 Functional Test Point PASS!!!
----[ 993325 ns] Number 8'd36 Functional Test Point PASS!!!
----[1020055 ns] Number 8'd37 Functional Test Point PASS!!!
----[1044205 ns] Number 8'd38 Functional Test Point PASS!!!
----[1053745 ns] Number 8'd39 Functional Test Point PASS!!!
----[1063265 ns] Number 8'd40 Functional Test Point PASS!!!
----[1072165 ns] Number 8'd41 Functional Test Point PASS!!!
----[1077305 ns] Number 8'd42 Functional Test Point PASS!!!
----[1101865 ns] Number 8'd43 Functional Test Point PASS!!!
----[1140075 ns] Number 8'd44 Functional Test Point PASS!!!
----[1178045 ns] Number 8'd45 Functional Test Point PASS!!!
----[1216255 ns] Number 8'd46 Functional Test Point PASS!!!
----[1245945 ns] Number 8'd47 Functional Test Point PASS!!!
----[1290205 ns] Number 8'd48 Functional Test Point PASS!!!
----[1336495 ns] Number 8'd49 Functional Test Point PASS!!!
----[1387255 ns] Number 8'd50 Functional Test Point PASS!!!
----[1388925 ns] Number 8'd51 Functional Test Point PASS!!!
----[1395085 ns] Number 8'd52 Functional Test Point PASS!!!
----[1401885 ns] Number 8'd53 Functional Test Point PASS!!!
----[1408365 ns] Number 8'd54 Functional Test Point PASS!!!
----[1415165 ns] Number 8'd55 Functional Test Point PASS!!!
----[1422595 ns] Number 8'd56 Functional Test Point PASS!!!
----[1429385 ns] Number 8'd57 Functional Test Point PASS!!!
----[1434365 ns] Number 8'd58 Functional Test Point PASS!!!
----[1460075 ns] Number 8'd59 Functional Test Point PASS!!!
----[1487665 ns] Number 8'd60 Functional Test Point PASS!!!
----[1507925 ns] Number 8'd61 Functional Test Point PASS!!!
----[1531075 ns] Number 8'd62 Functional Test Point PASS!!!
----[1544165 ns] Number 8'd63 Functional Test Point PASS!!!
----[1558035 ns] Number 8'd64 Functional Test Point PASS!!!
----[1587035 ns] Number 8'd65 Functional Test Point PASS!!!
----[1614255 ns] Number 8'd66 Functional Test Point PASS!!!
----[1629675 ns] Number 8'd67 Functional Test Point PASS!!!
----[1644675 ns] Number 8'd68 Functional Test Point PASS!!!
============================================================
Test end!
----PASS!!!
$finish called at time : 1645135 ns : File "D:/Architecture/lab1/
↪  lab3/CPU_CDE/mycpu_verify/testbench/mycpu_tb.v" Line 267
```

程序	mycpu 结束	gettrace 结束
func_lab6（42 点）	1.078 ms	1.194 ms
func_lab7（68 点）	1.645 ms	1.836 ms

5.2 乘除法器模块级测试

封装见第 4.3 节。

mul_tb 每拍更换一对随机操作数。参考模型按 mul_signed 把操作数扩成 33 位再相乘，并与打了一拍的输入比较。10 µs 内 991 次比较全部为 OK=1。有符号 $-2031628787 \times -1716266445 = 3486816315824152215$，无符号 $4052285667 \times 1453815725 = 5891276624876713575$。

div_tb 的原上限是 34 拍。无符号除法在第 34 拍给出 complete: 465 ns 时 $2097015289 \div 3812041926$ 的商为 0、余数为 2097015289，s_OK 与 r_OK 均为 1，同一拍测试台因到达上限而结束。有符号除法在第 36 拍完成。上限改为 60 拍后，同一套随机激励在 50 µs 内完成 129 次除法，商和余数全部与参考模型一致：无符号 67 次，每次 34 拍；有符号 62 次，每次 36 拍。$-442812725 \div -217507098$ 的商为 2、余数为 -7798529；$2087561720 \div -607297353$ 的商为 -3、余数为 265669661。两例的余数符号都与被除数相同。

5.3 波形

波形从仿真 VCD 里截取短窗口，用 matplotlib 画出。时钟周期 10 ns。单周期脉冲的格子里不标十六进制，数值写在箭头旁。

5.4 SLTIU

func_lab7 中 bfc4ab28 是 sltiu $v0,$t0,-2967。$t0 = c255e3d8，立即数符号扩展为 fffff469。无符号比较成立，写回 1。若把立即数零扩展成 0000f469，c255e3d8 更大，结果应是 0。图 1 里访存级和写回都是 1。

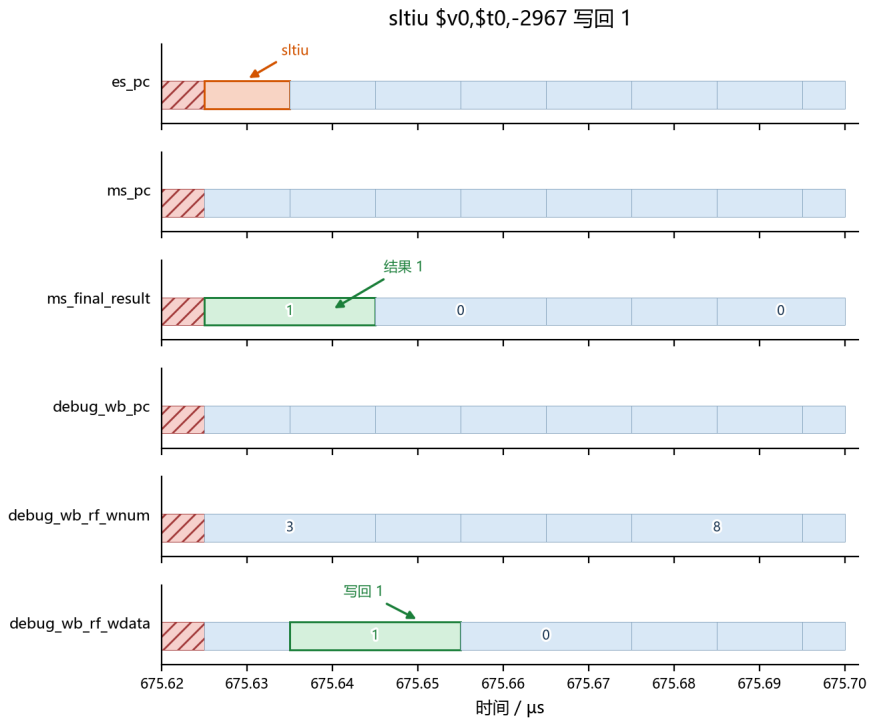


图 1: `sltiu` 在 675.63 µs 进入执行级，下一拍访存结果为 1，写回 1。

5.5 ANDI

`bfc80c58` 是 `andi $v0,$t0,0x5e89`，`$t0 = 4c23cdd4`。零扩展后与出来是 `4c80`。同一立即数若符号扩展成 `ffff5e89`，结果是 `4c234c80`。图 2 的写回是 `4c80`，和测试事先放入 `$v1` 的参考值相同，随后的 `bne` 不跳转。

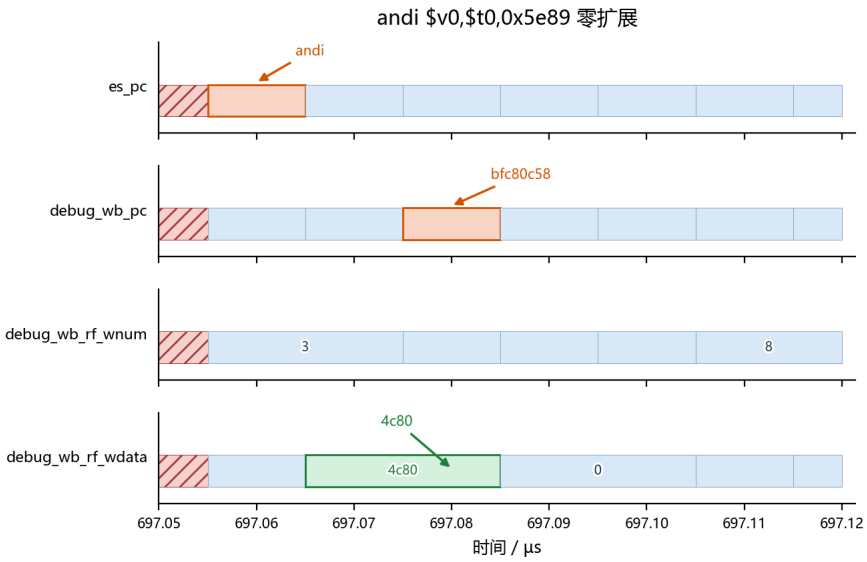


图 2: `andi` 写回 `4c80`。`debug_wb_pc` 为 `bfc80c58` 的那一拍与写回数据对齐。

5.6 MULT 与 MFLO

`bfc85118` 的 `mult $t0,$t1` 中，`$t0 = 45b90738`，`$t1 = d70d64f0`。执行级 PC 停在这条指令上，`es_ready_go` 低了两拍（20 ns）。993.675 µs HI 变成 `f4d903bb`，LO 变成 `a20a480`，执行级同时放行。紧跟着的 `mflo $s5` 把 LO 写入通用寄存器，见图 3。除法的停顿与此相同：有符号核 36 拍、无符号核 34 拍后 `dout` 有效，`es_ready_go` 保持为 0，直到 HI/LO 写完。`func_lab6` 的 n35–n42 已通过。

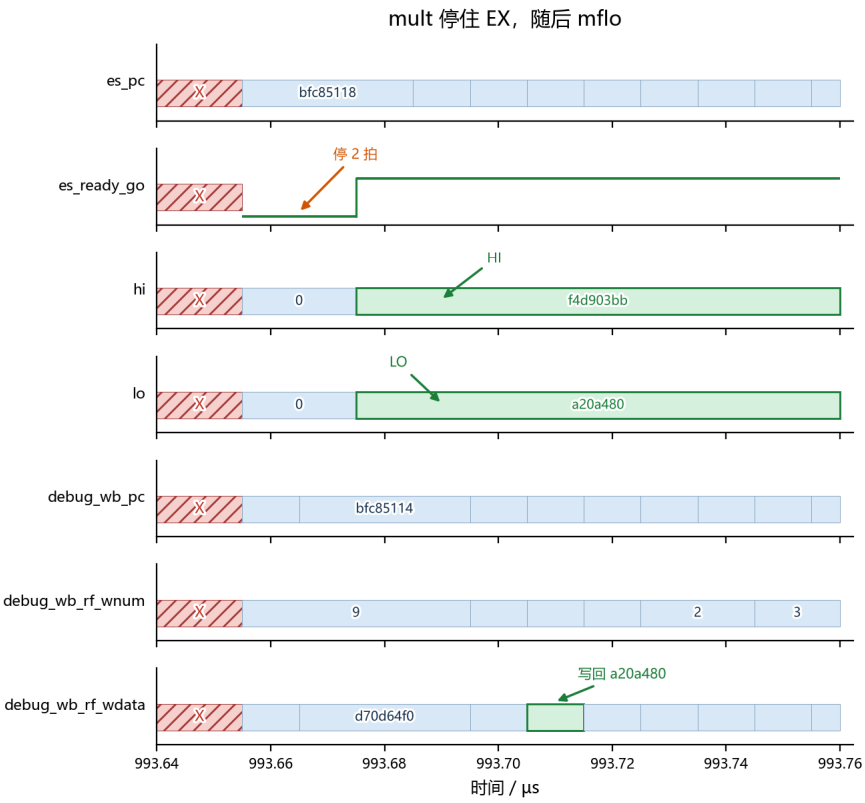


图 3: `mult` 停两拍后更新 HI/LO，`mflo` 写回 `a20a480`。

5.7 分支条件成立的 BGEZ

`bfc60024` 是 `bgez $t0`，目标，`$t0 = 44f2bff4`，条件成立。图 4 里执行级先是这条分支，译码级是延迟槽 `bfc60028` 的 `nop`，再下一拍 `ds_pc` 变为 `bfc60000`，顺序地址 `bfc6002c` 被跳过。

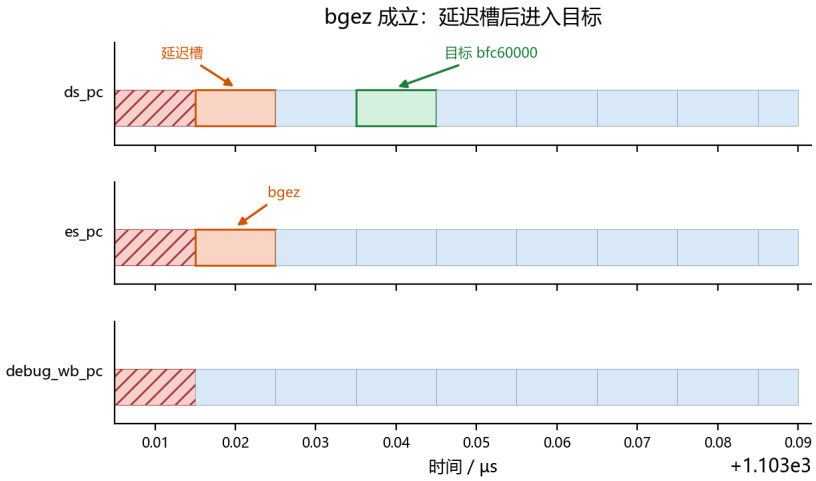


图 4: 分支条件成立的 bgez：延迟槽之后取目标 bfc60000。

5.8 分支条件不成立的 BLTZAL

bfc40694 是 bltzal $t0，目标为 bfc40670。$t0 = 655e408b，符号位为 0，br_taken 保持 0。延迟槽 bfc40698 之后，ds_pc 进入顺序地址 bfc4069c。同一条指令把 PC+8 = bfc4069c 写入 $31，见图 5。

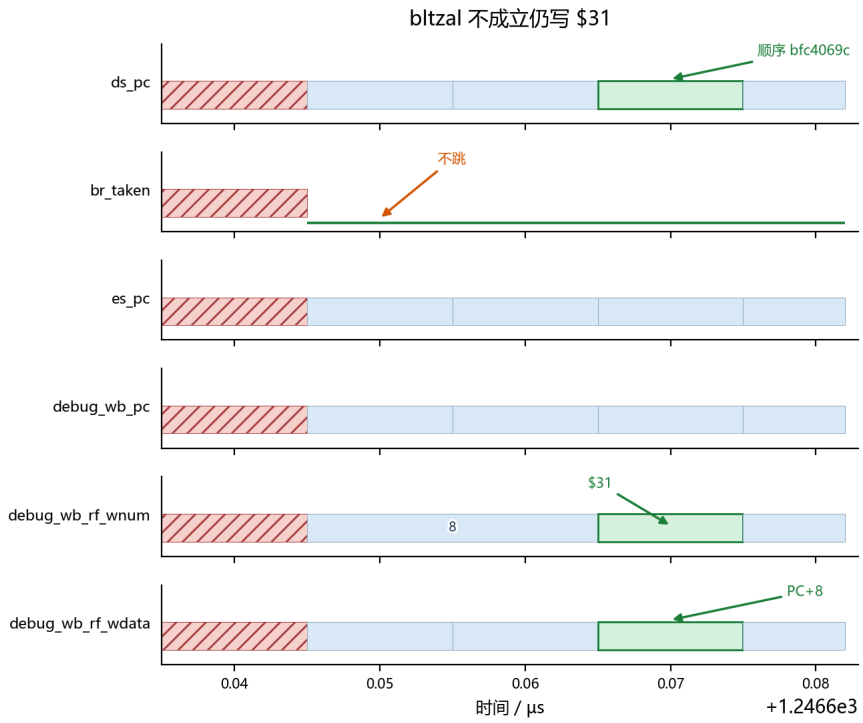


图 5: 分支条件不成立的 `bltzal` 不改变 PC，写回寄存器号为 31，数据为 `PC+8`。

5.9 非对齐 LWR

`bfc9a264` 就是前面写回不一致的那条 `lwr`。地址 `800dbd99` 的低两位为 1，读回的字是 `5c049582`，锁存的 `rt` 旧值是 `ca18f110`。拼接结果 `ca5c0495` 出现在访存级，并作为 `$v0` 的写回，见图 6。修正前这一拍写回的是 `f15c0495`。

同组 `LWL`（n63）在这次修正前已经通过。例如 `bfc5beac` 的地址低两位为 2，按 `{mem[23:0], rt[7:0]}` 拼成 `e8d3ee80`，与测试参考值一致。

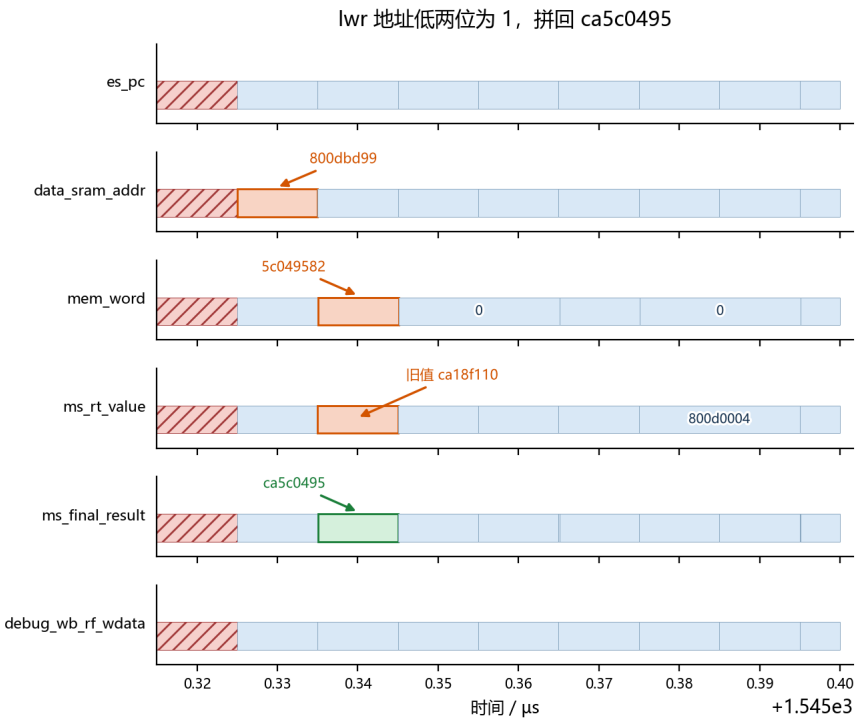


图 6: `lwr`：地址低两位为 1，旧值高字节 `ca` 与内存 `[31:8]` 拼成 `ca5c0495`。

5.10 SB

`bfc8e15c` 是 `sb $t1, 25068($t0)`。地址 `800d78d8` 的低两位为 0。`data_sram_wen` 为 1，只打开最低字节，写数据把 `0a` 复制四次。原字 `b4f00930` 变成 `b4f0090a`，后面的 `lw` 读回同一数值，见图 7。图中稍后 `wen = f` 的那一拍是测试里的 `sw`，用来写相邻的字。

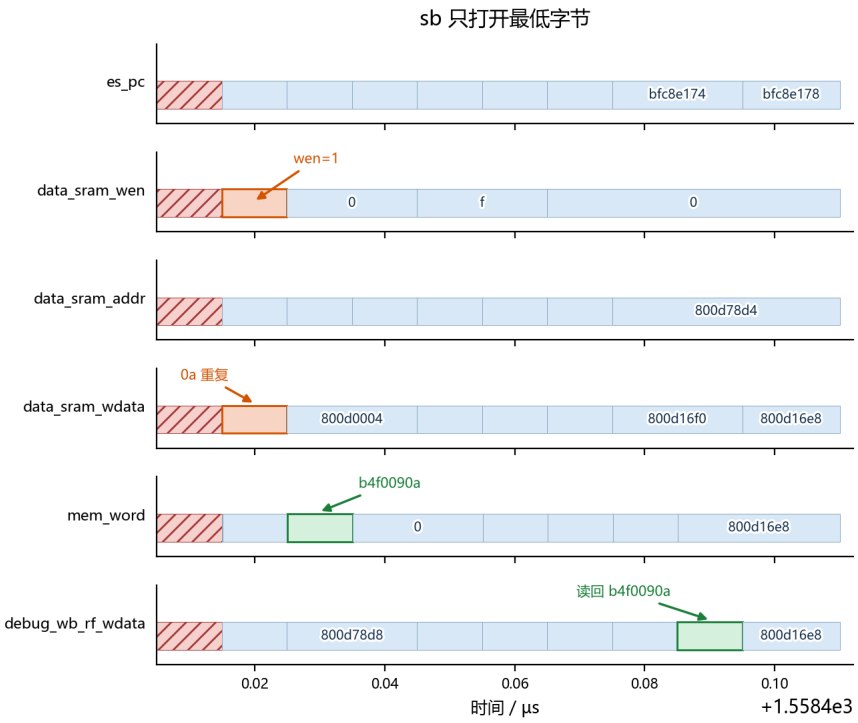


图 7: sb 的写使能为 1。内存字变为 b4f0090a，随后 lw 读回该值。

6 实验收获

流水线从实验 1 的定点算术扩展到乘除、带链接的分支和部分字访存。立即数按指令选择符号扩展或零扩展，SLTIU 在符号扩展之后做无符号比较。乘除把结果写入 HI/LO 时让执行级停住，后面的 MFHI/MFLO 读到的是新值。模块级随机测试里，乘法 991 次与参考值一致；除法的无符号核在 34 拍给出结果，与测试台原上限重合，上限放宽到 60 拍后 129 次商和余数一致。LWR 第一次失败时写回只差一个高字节，对照地址低两位和拼接表可以找到被截断的那一段。

7 附：源码目录

CPU 仍在实验 1 的工程里改。lab2 放测试程序、仿真脚本和本报告。

- lab1/lab3/CPU_CDE/mycpu_verify/rtl/myCPU/ID_stage.v
- lab1/lab3/CPU_CDE/mycpu_verify/rtl/myCPU/EXE_stage.v
- lab1/lab3/CPU_CDE/mycpu_verify/rtl/myCPU/MEM_stage.v
- lab1/lab3/CPU_CDE/mycpu_verify/rtl/myCPU/mycpu.h

- lab1/lab3/CPU_CDE/mycpu_verify/testbench/mycpu_tb.v
- lab1/lab3/CPU_CDE/cpu132_gettrace/golden_trace.txt
- lab1/lab3/CPU_CDE/soft/func_lab6/obj/inst_ram.coe
- lab1/lab3/CPU_CDE/soft/func_lab7/obj/inst_ram.coe
- lab2/lab6/func_lab6/
- lab2/lab7/func_lab7/
- lab2/lab6/乘法器模块级验证顶层/mul.v
- lab2/lab6/乘法器模块级验证顶层/div.v
- lab2/sim/run_md_tb.tcl
- lab2/sim/run_mycpu_lab6.tcl
- lab2/report/report.tex